6. a. Multi – level linked list

–Delete At End

–Program Analysis(Main List) and Time Complexity

***Program* : –**

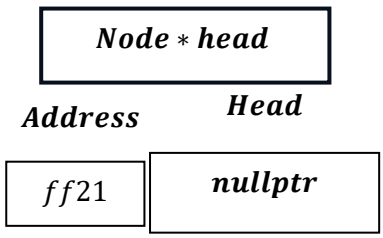
```
void deleteAtEnd()
{
    if (head == nullptr)
        return;

    if (head->next == nullptr)
    {
        free(head);
        head = nullptr;
        return;
    }

    Node *temp = head;
    while (temp->next->next != nullptr)
    {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = nullptr;
    cout << "Deleted from end." << endl;
}
int main()
{
    int choice;
    while (true)
    {
       cout << "6. Delete at End (Main)" << endl;
        cin >> choice;
```

```
        switch (choice)
        {
       case 6:
            deleteAtEnd();
            break;
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Program Analysis

A. When List is Empty

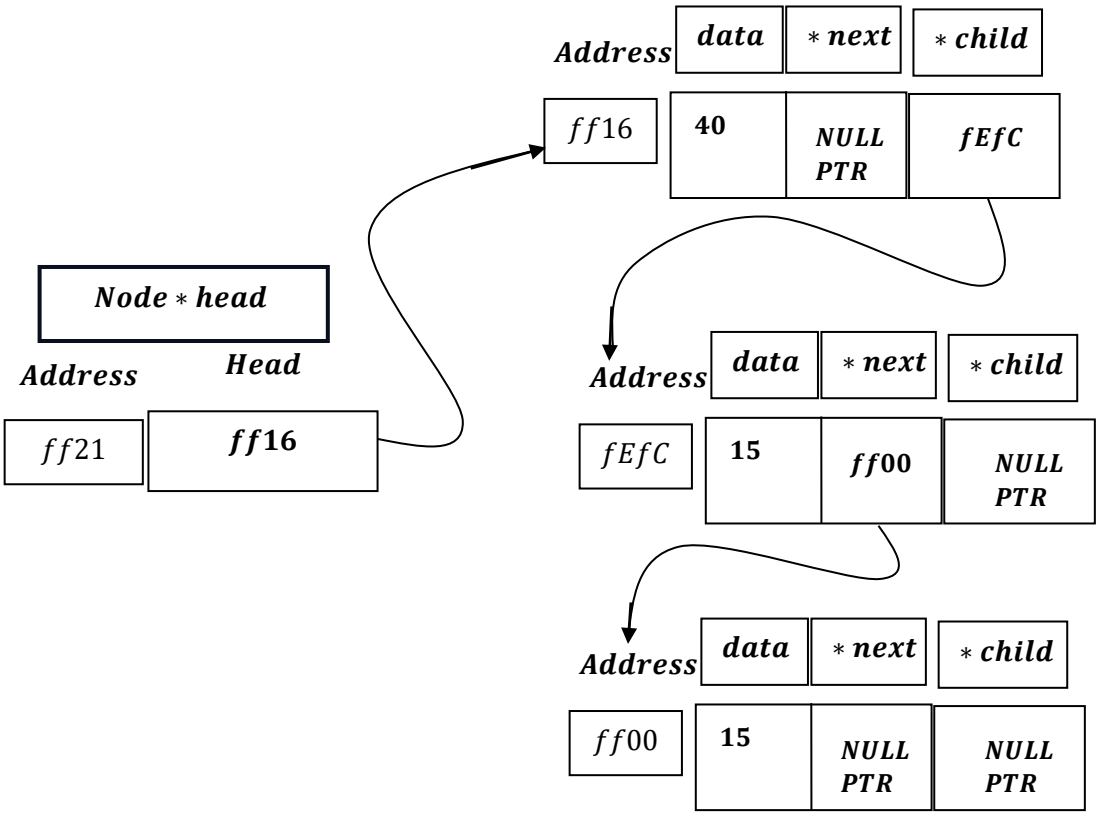
```
    if (head == nullptr)
       return;
```



When head = nullptr, means ``List is Empty.``

``Return void`` and exit from the function.

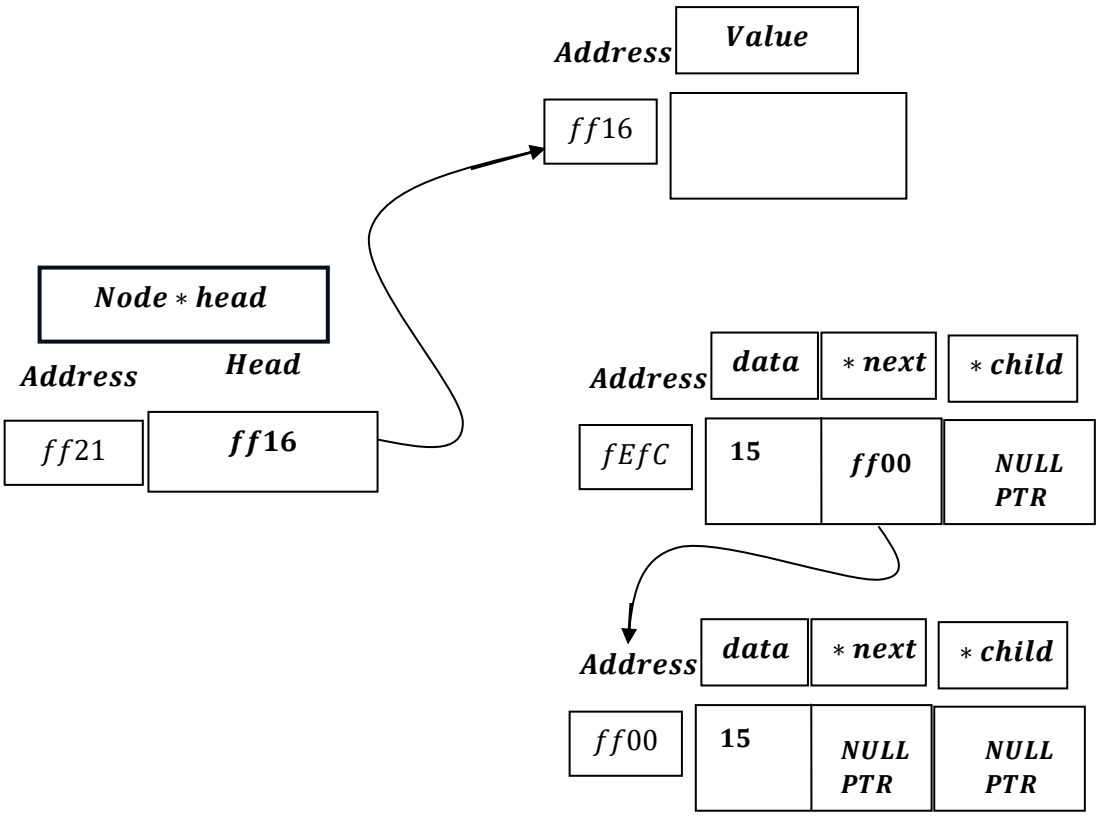
B. When List has single node.



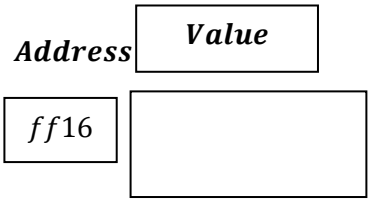

```
if (head->next == nullptr)
{
    free(head);
    head = nullptr;
    return;
}
```

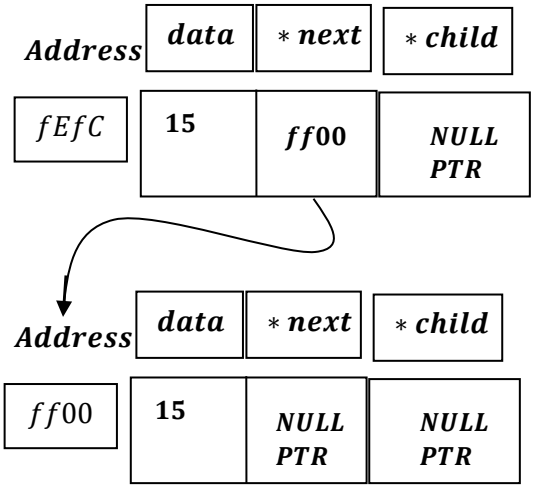
Here head → next = NULLPTR, therefore:

$Free(head: ff16)$




```
        head = nullptr
```

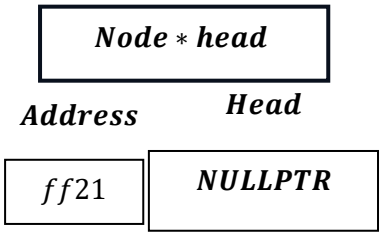




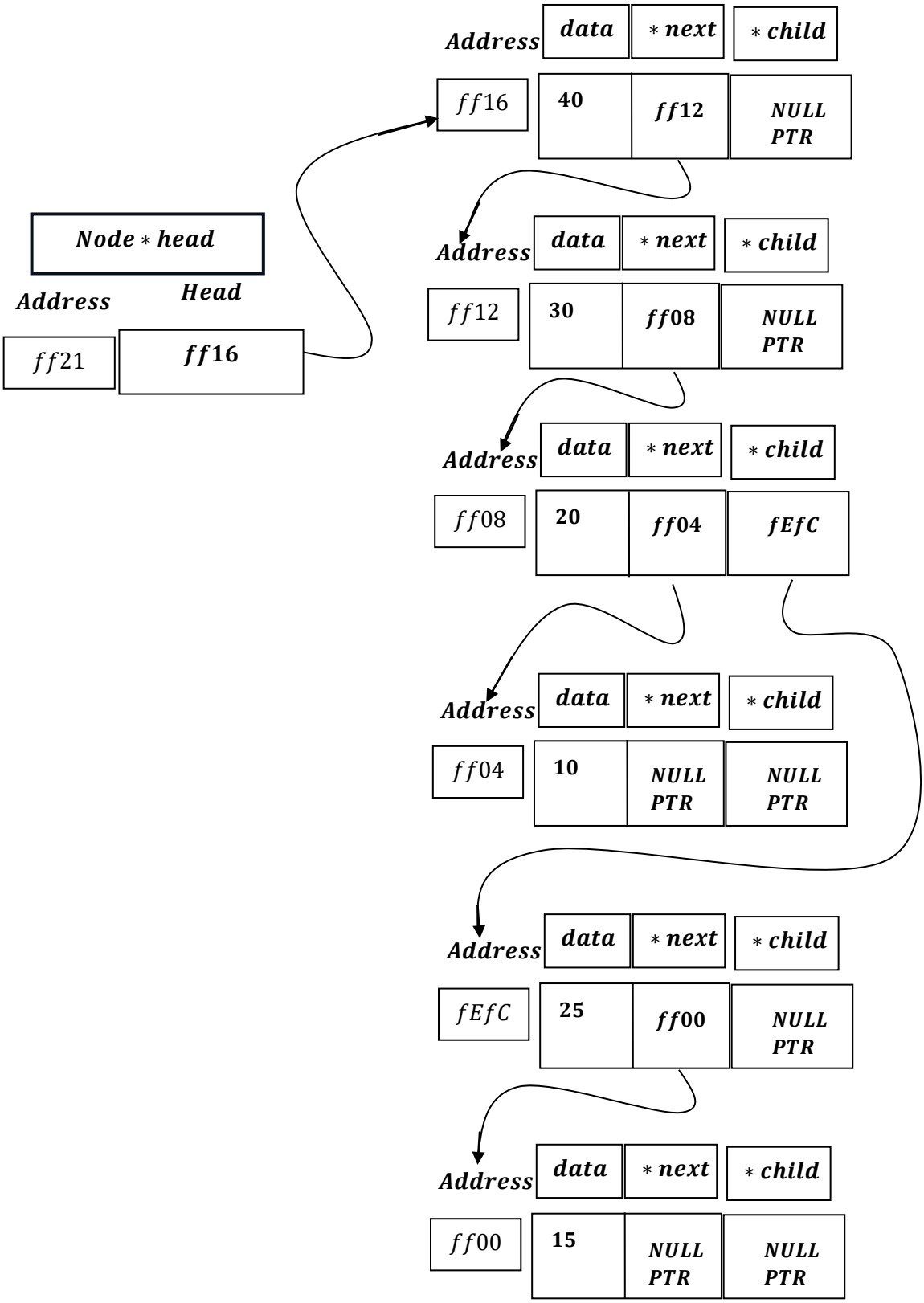

```
        return;
```

Return void and exit from function.

Now, list is empty.

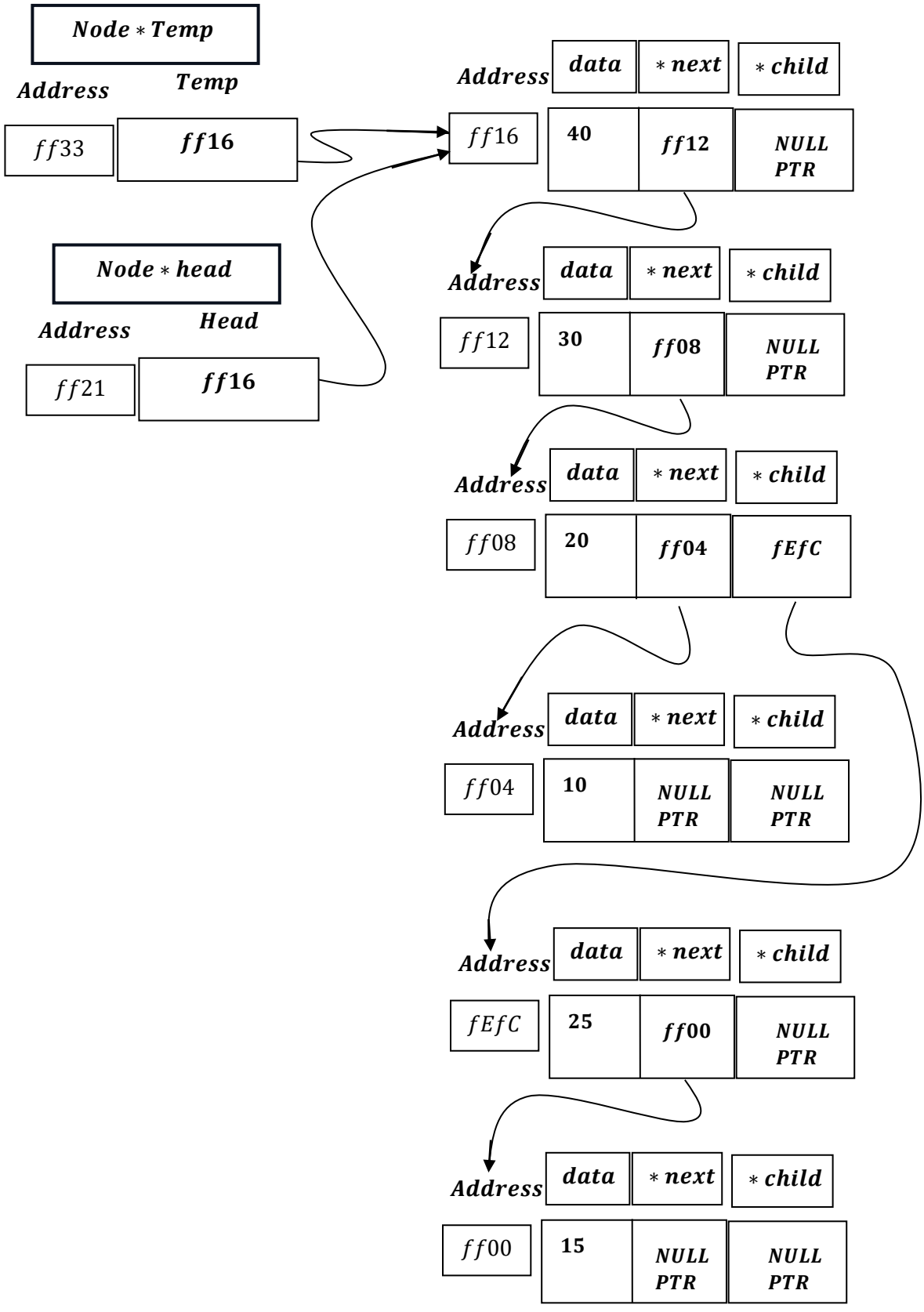


C. When List has numerous node.



```
    Node *temp = head;
```

$Node * temp = head : ff16.$

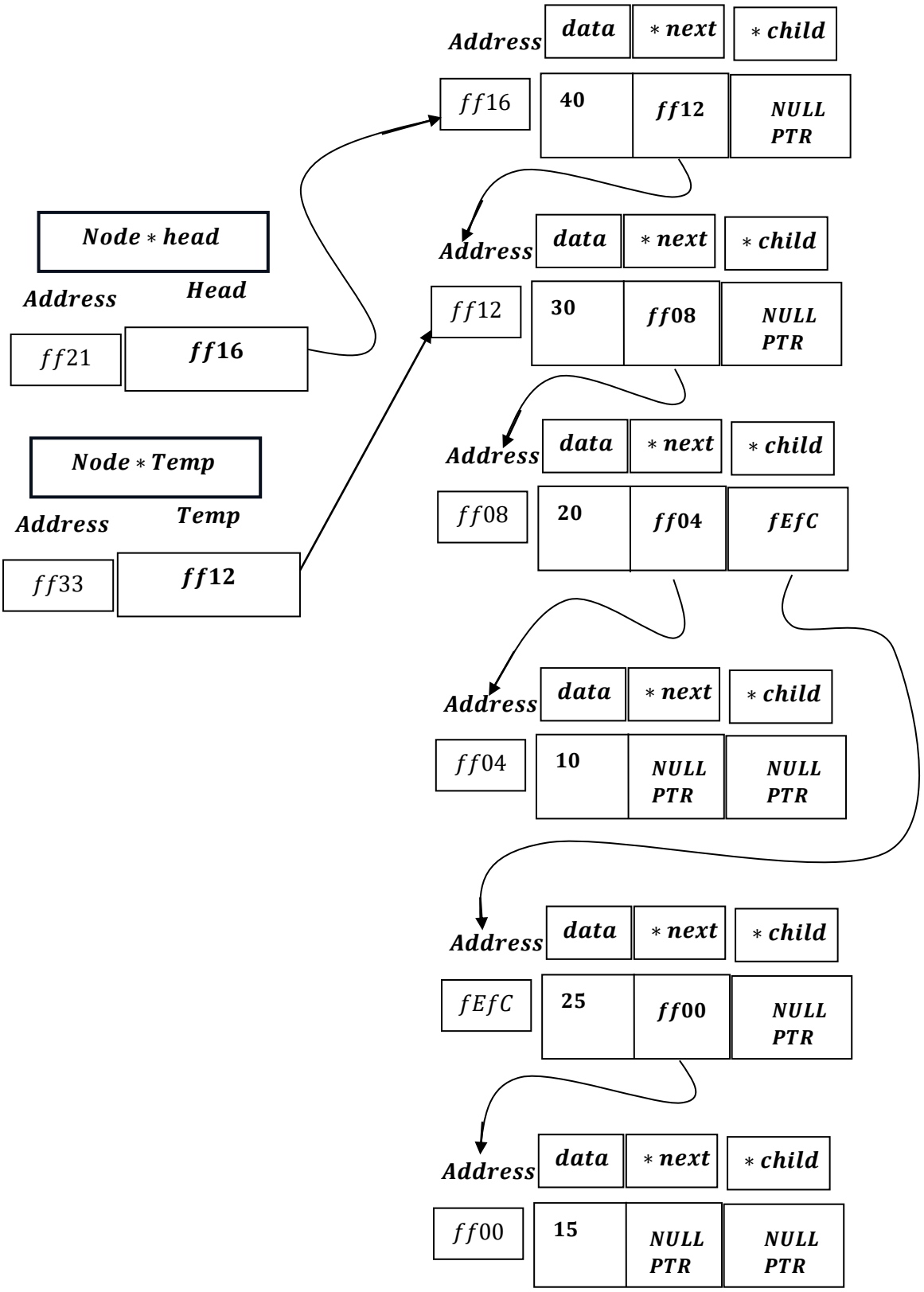


```
    while (temp->next->next != nullptr)
    {
        temp = temp->next;
    }
```

It will point to previous node of end node.

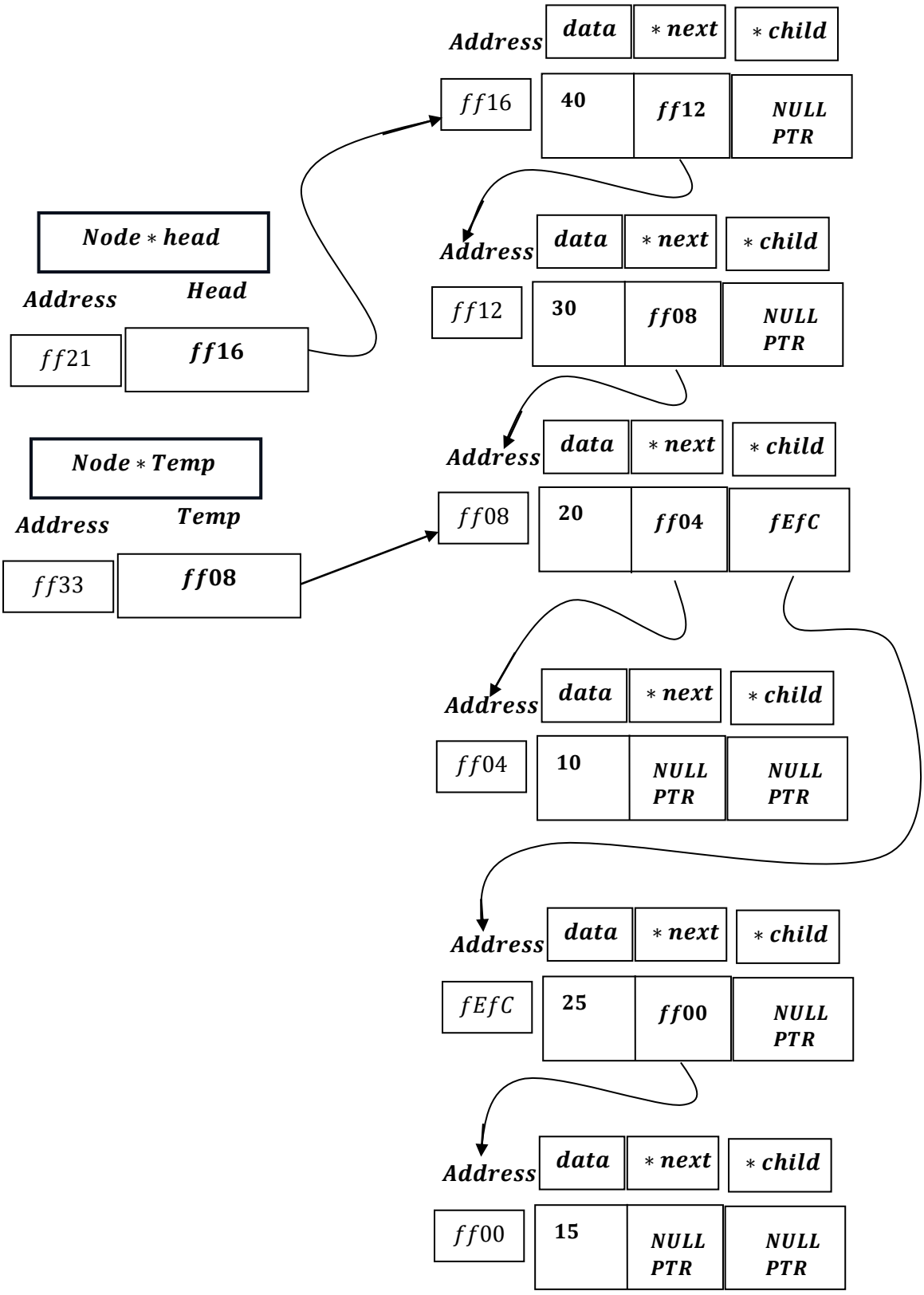
$$\boldsymbol{TEMP \rightarrow NEXT \rightarrow NEXT: ff08 \neq NULLPTR:}$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT : ff12}$$


Address
data
* next
* child
ff16
40
ff12
NULL PTR
Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Node * Temp
Address
Temp
ff33
ff12
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR

$$TEMP \rightarrow NEXT \rightarrow NEXT: ff04 \neq NULLPTR:$$

$$TEMP = TEMP \rightarrow NEXT : ff08$$

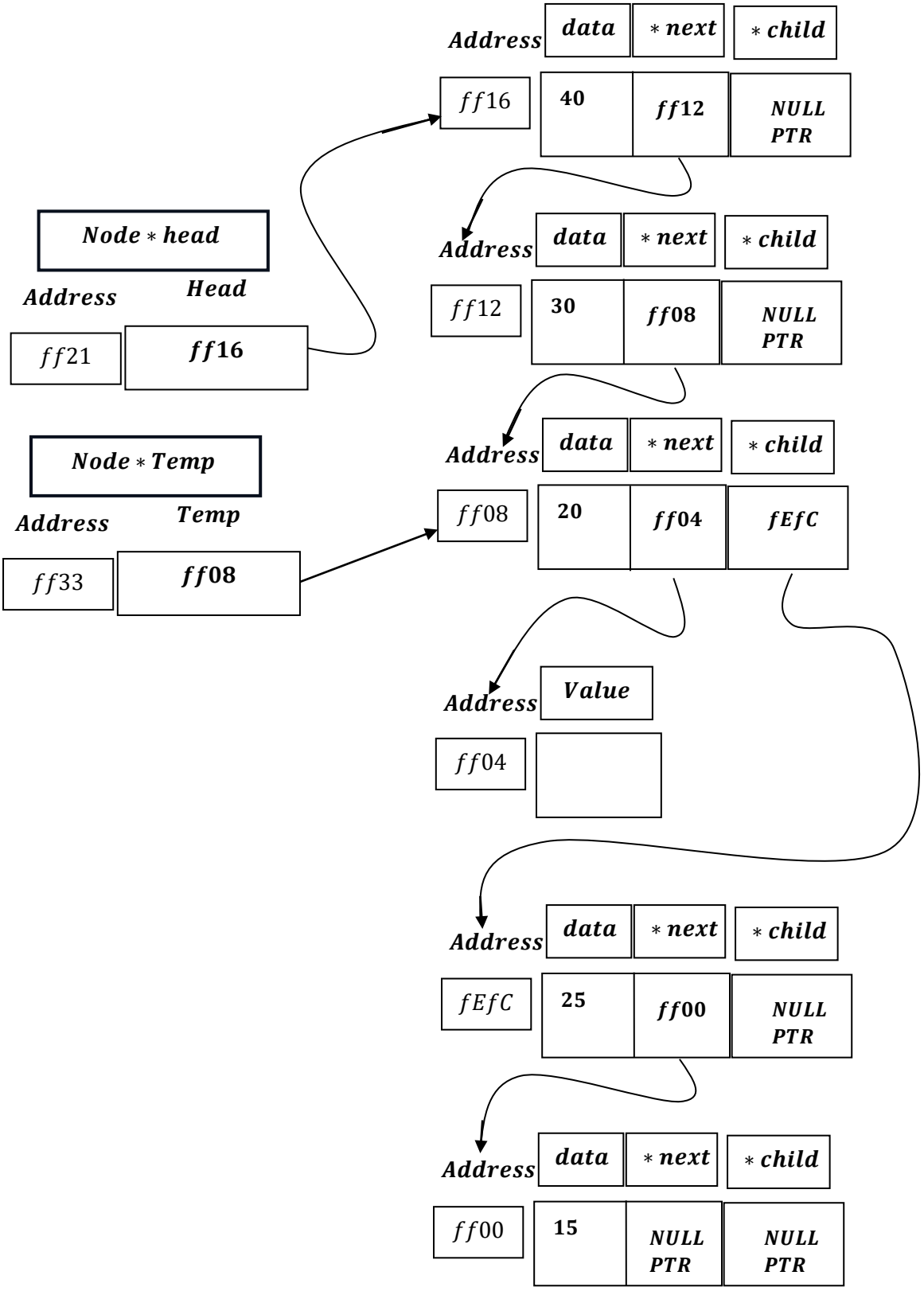


$$TEMP \rightarrow NEXT \rightarrow NEXT : NULLPTR \neq NULLPTR \begin{bmatrix} Condition \\ become \\ false \end{bmatrix}$$

And loop exits.

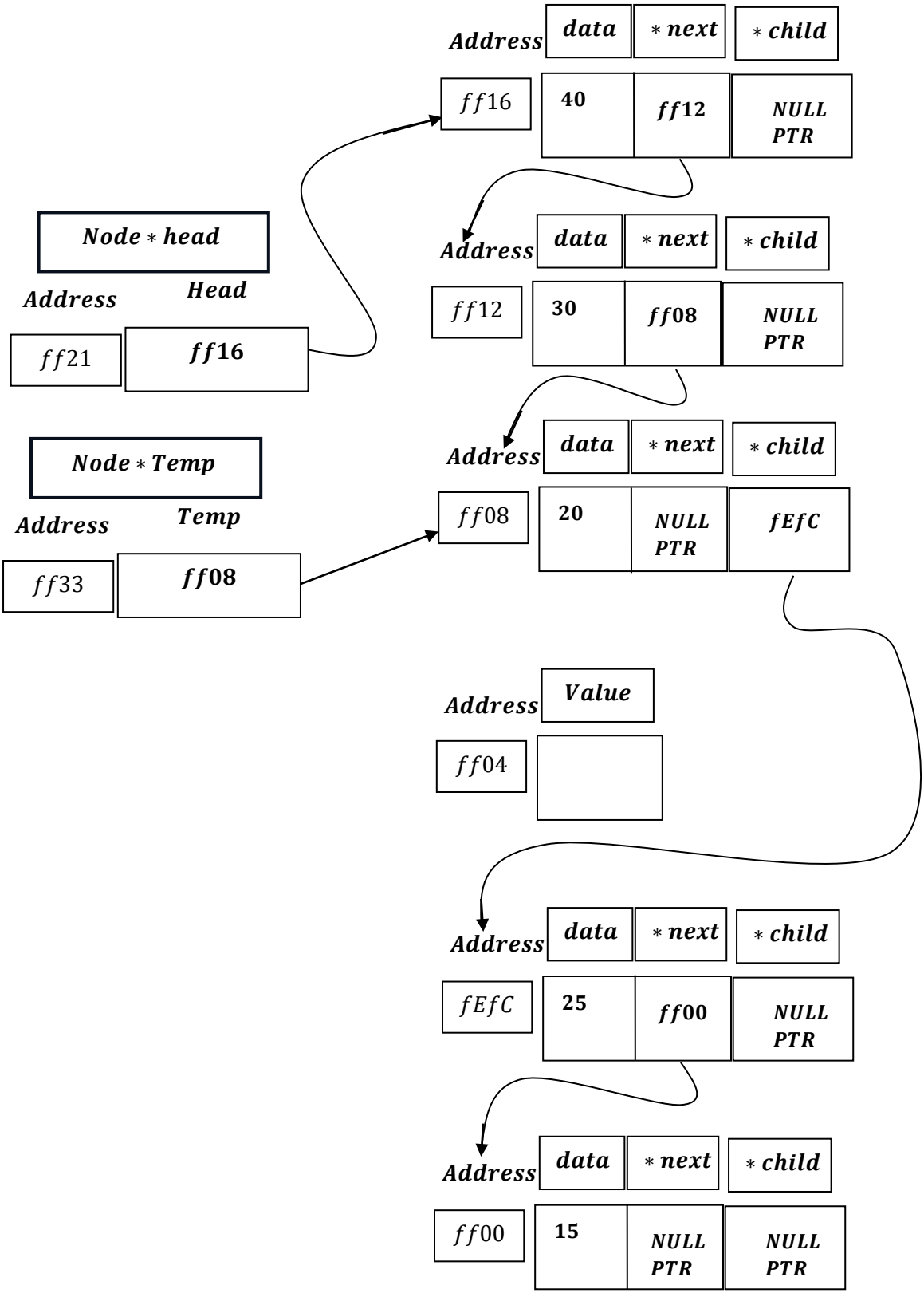
```
    free(temp->next);
```

$$free(temp \rightarrow next : ff04)$$


Address
data
* next
* child
ff16
40
ff12
NULL PTR
Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Node * Temp
Address
Temp
ff33
ff08
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
Value
ff04
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR

```
    temp->next = nullptr;
```

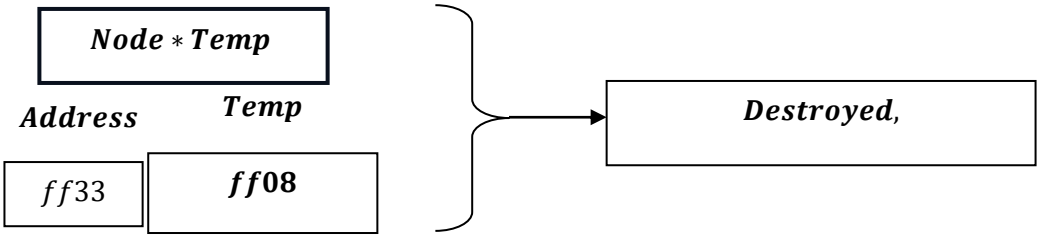
$TEMP \rightarrow NEXT = NULLPTR;$



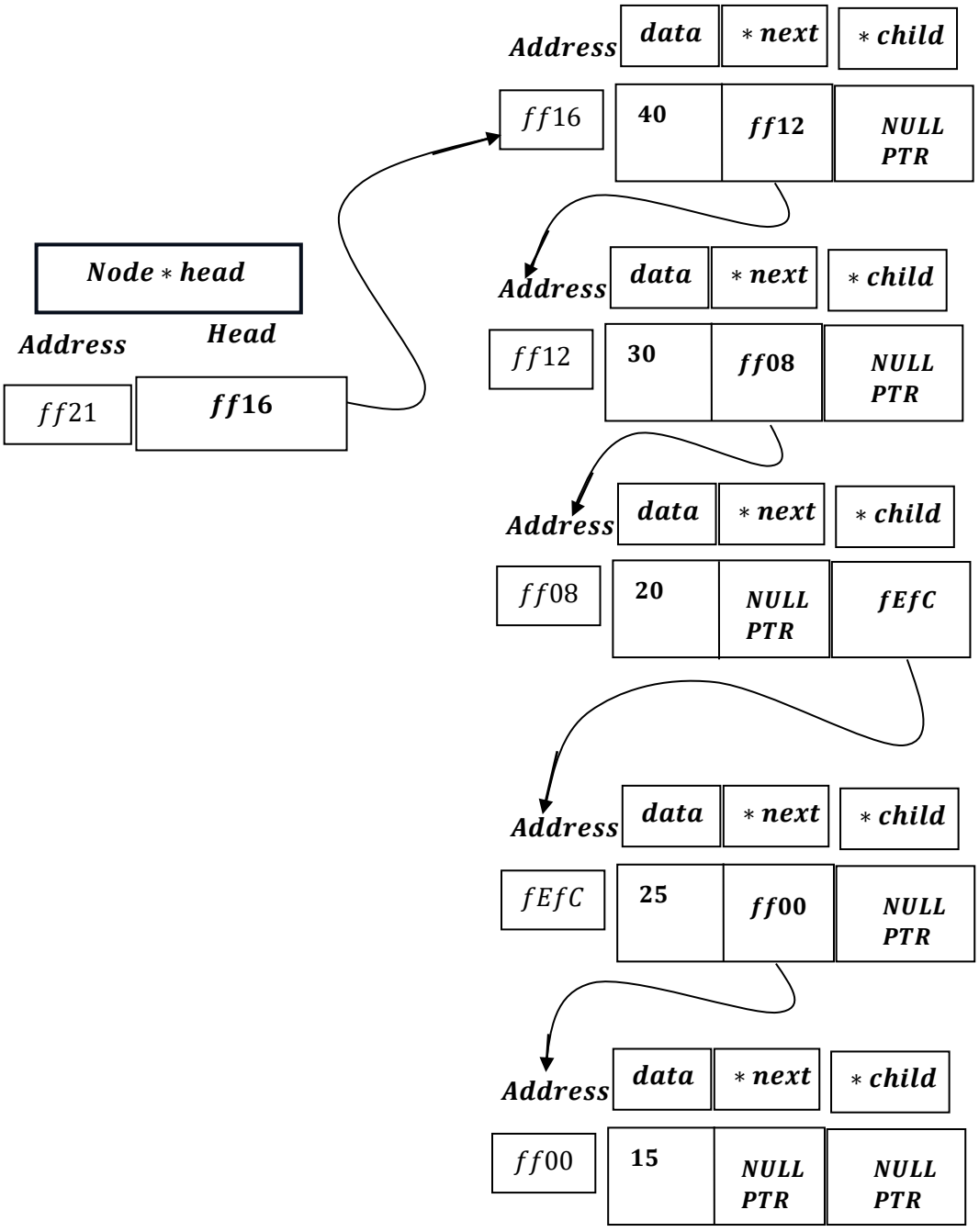
```
    cout << "Deleted from end." << endl;
```

***Print*: ``*Deleted from end.*``**

And, as the function gets finished.



***And we are left with*:**


Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
NULL PTR
fEfC
Address
data
* next
* child
fEfC
25
ff00
NULL PTR
Address
data
* next
* child
ff00
15
NULL PTR
NULL PTR

Time Complexity

Best Case

1. a. ***Absolute Best Case : When List is Empty :*** $\Omega(1)$

1. b. ***Not Absolute Best Case but can be considered as Best Case , when List has a single node .*** $\Omega(1)$

Worst Case:

When List has `n` nodes in Main List and we have to delete the n^{th} ***node of Main List .***

The inner loop statement runs: $`n-2`$ ***times , as it will point to the 2nd last node of the Main List.***

Other assignments and freeing memory takes constant time : $O(1)$.

Hence : $O(n-2)+O(1)=O(n)$ ***time complexity in Worst Case Scenario.***

Average Case

There is no true average case , as either the program will execute its best case or worst case , hence $Average\ Case = Worst\ Case = \Theta(n)$.
